Pimpri Chinchwad Education Trust's Pimpri Chinchwad College of Engineering
(An Autonomous Institute Affiliated to SPPU, Pune)

DevOps - Continuous Integration Laboratory Assignment 6

Student Name: Rutuja Jagtap PRN: 123B1B146 Batch: C1
Department: Computer Engineering Academic Year: 2026-2027 Semester: I

Integrate Jenkins with a Git Repository and Automate Source Code Retrieval from GitHub

Continuous Integration, Jenkins Automation Server, Git Plugin, SCM Checkout, GitHub Webhooks, Automated Build Triggers, and Pipeline As Code

1. Aim

To configure Jenkins with a GitHub repository and automate the process of pulling source code from the repository for Continuous Integration.

2. Problem Statement

Manual code pulling, local environment compilation, and unsynchronized testing create integration bottlenecks and lead to late discovery of build errors in collaborative software engineering. The task is to establish an automated Continuous Integration pipeline by connecting an enterprise Jenkins automation server with a remote GitHub repository, enabling automated source code checkout upon branch commits and executing automated build verification within an isolated Jenkins workspace.

3. Learning Objectives

Area	Learning Objective
Continuous Integration Fundamentals	Understand the role and lifecycle of Jenkins in automated CI/CD software pipelines.
Plugin Management	Install, configure, and verify the Git Plugin, GitHub Plugin, and Pipeline Plugin in Jenkins.
SCM Integration	Connect Jenkins securely to remote GitHub repositories via HTTPS, Personal Access Tokens (PAT), or SSH credentials.
Automated Source Code Checkout	Create and configure Jenkins Freestyle and Declarative Pipeline jobs for automated branch checkout.
Build Execution & Verification	Execute automated build tasks, inspect console outputs, and verify workspace integrity.
DevOps Feedback Loops	Understand webhooks and SCM polling mechanisms to establish rapid, event-driven developer feedback loops.

4. Continuous Integration and Jenkins Overview

Continuous Integration (CI) is a foundational software development practice where team members integrate code changes into a shared central repository frequently—often multiple times a day. Each integration is verified by an automated build, static code analysis, and automated test suite, enabling teams to detect merge conflicts and integration regressions early before code progresses to staging and production.

Jenkins is an open-source, extensible automation server designed to orchestrate continuous integration and continuous delivery workflows. Written in Java, Jenkins operates on a master-agent architecture and boasts a massive plugin ecosystem.

By integrating Jenkins with GitHub, every code commit pushed to remote repositories automatically triggers source code retrieval, compiles project binaries or validates script syntax, runs automated unit test suites, and generates comprehensive build reports for developers.

5. Jenkins Architecture, Git SCM Concepts, and Integration Workflow

The Jenkins-GitHub CI workflow follows a closed feedback loop: developers commit code changes to GitHub, Jenkins detects updates via webhooks or SCM polling, checks out the exact commit hash into a segregated workspace, executes build instructions, and records build telemetry.

5.1 End-to-End CI Architecture & Workflow

The operational workflow connects three primary nodes: the developer workstation, the remote GitHub repository hosting the application code, and the Jenkins automation server listening on port 8080. When a build is triggered, the Jenkins Git SCM plugin executes git fetch and git checkout to clone the repository into the project workspace directory.

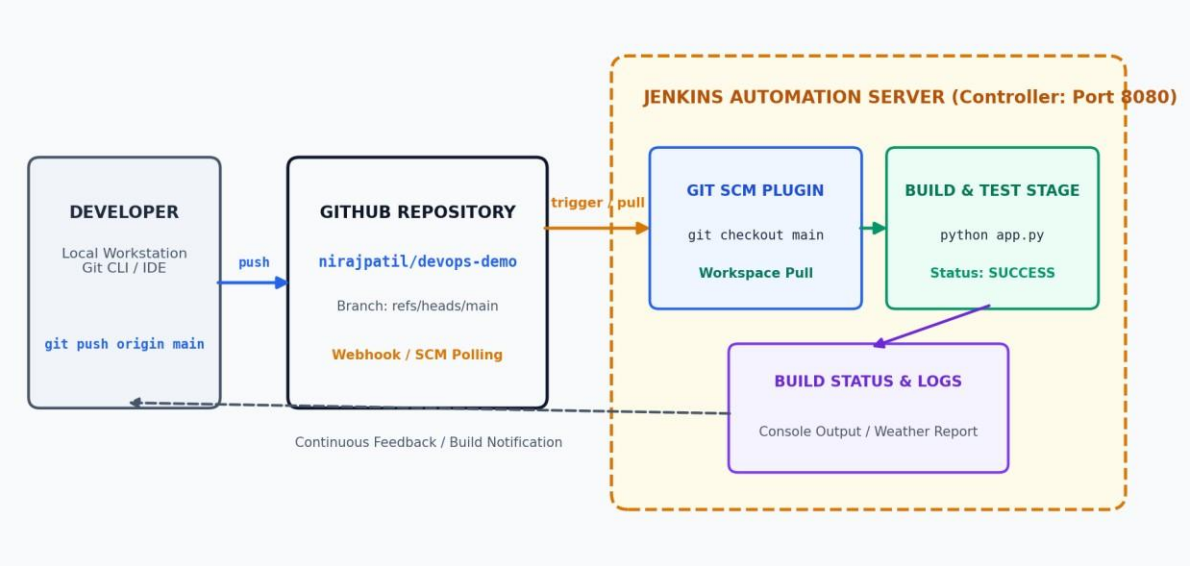


Figure 1. Architecture and operational workflow of Jenkins integration with GitHub

5.2 Core CI Building Blocks

Component / Block	Purpose	Implementation in this Practical
Jenkins Controller	Central orchestrator managing jobs, plugins, security, and workspaces.	Local Jenkins service running on port 8080.
Git Plugin	Provides Source Code Management (SCM) capabilities to pull Git repositories.	Jenkins Git Plugin v5.2.2 integrated with git.exe.
GitHub Plugin	Enables repository URL parsing, webhooks, and push event triggers.	GitHub Integration Plugin v1.39.0.
Freestyle Job	Standard GUI-configurable Jenkins project for sequential tasks.	Project 'GitHub-Jenkins-Demo' tracking branch '*/main'.
Jenkinsfile (Pipeline)	Declarative Pipeline-as-Code script defining multistage execution.	Declarative pipeline with Checkout, Build, and Test stages.
Workspace Directory	Dedicated filesystem folder where repository code is cloned and built.	C:\ProgramData\Jenkins\.jenkins\workspace\Git Hub-Jenkins-Demo.
Console Output	Real-time streaming log of command execution and Git SCM events.	Displays Git checkout commit SHA and Python build status.

5.3 Operational Principles of Automated SCM Retrieval

- Decoupled Source Control: Code resides centrally on GitHub, ensuring the build system pulls the authoritative repository state rather than local developer copies.
- Immutable Commit Referencing: Jenkins records the precise Git commit SHA-1 hash for each build, guaranteeing complete traceability and auditability.
- Clean Workspace Isolation: Each project is allocated an isolated workspace directory, preventing cross-project dependency pollution.
- Automated Verification Gates: Build failures immediately halt downstream pipeline stages, protecting mainline branches from broken code.

5.4 Pipeline Execution Lifecycle

In modern CI/CD practice, automated retrieval is codified inside a declarative Jenkinsfile. The pipeline progresses sequentially across distinct stages: Trigger, SCM Checkout, Build, Test, and Status Notification.

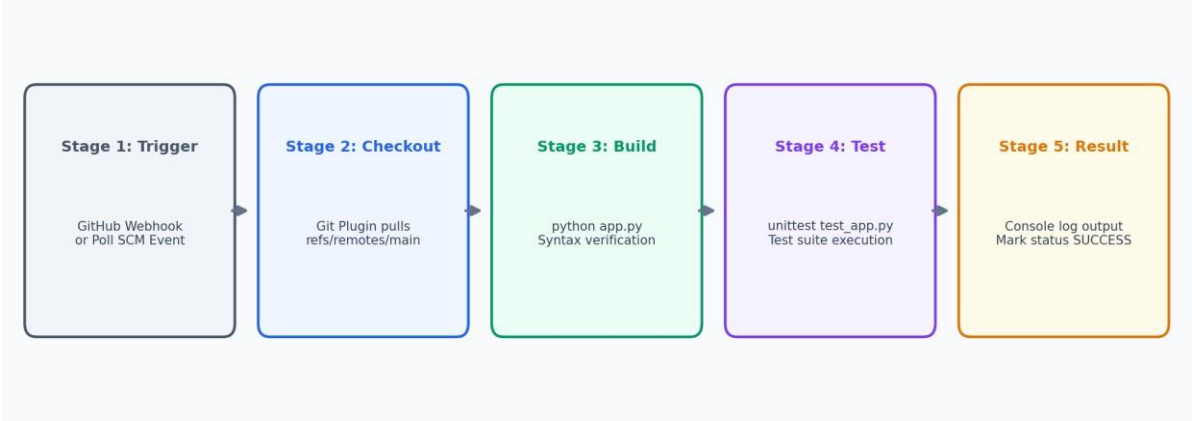


Figure 2. Declarative Jenkins pipeline execution lifecycle

6. Installation, Prerequisites, and Environment Configuration

The practical requires verifying system prerequisites including Java runtime, Git CLI, and the running Jenkins automation server.

6.1 Prerequisites Matrix

Prerequisite Component	Minimum Required Version	Status & Verification Point
Java Development Kit (JDK)	Java 17 or Java 21 LTS	Temurin-17.0.12 verified via java -version.
Git CLI	Git 2.40.0+	Git 2.46.2 installed and configured in system PATH.
Jenkins Automation Server	Jenkins 2.440+ LTS	Active Jenkins instance listening on http://localhost:8080.
GitHub Account & Repository	GitHub Public / Private Repo	Remote repository 'devops-demo' created with main branch.

6.2 System Verification Commands

Verify the presence and responsiveness of Git, Java, and Jenkins using terminal commands: **Prerequisites Verification**

```
git --version java
-version
curl -I http://localhost:8080/login
```

6.3 Jenkins Plugin Configuration

To enable Git connectivity, navigate to Manage Jenkins -> Plugins -> Available Plugins and ensure the following plugins are active:

- Git Plugin: Core provider for Git SCM integration and shallow cloning.
- GitHub Plugin: Adds repository linking, webhook endpoints, and status publishing.
- Pipeline Plugin: Enables execution of Jenkinsfile declarative pipelines.

7. GitHub Repository and Sample Application Structure

A dedicated GitHub repository named 'devops-demo' is established containing a sample web application, unit tests, frontend landing page, and project documentation.

7.1 Repository Directory Structure

File Name	Role in Application	Content Overview
README.md	Project Documentation	Explains project overview, installation, and CI/CD workflow.
index.html	Frontend Presentation	Lightweight HTML5 landing page for the application.
app.py	Application Source Code	Python Flask REST API with / and /health status endpoints.
test_app.py	Automated Test Suite	Unit test cases asserting HTTP status 200 on all routes.
Jenkinsfile	Pipeline as Code	Declarative pipeline codifying checkout, build, and test stages.

7.2 Application Implementation (app.py) **devops-demo/app.py**

```
import os
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/') def
home():     return
jsonify({
        'status': 'success',
        'message': 'DevOps CI/CD Demo Application running successfully!',
        'ci_tool': 'Jenkins Automation Server',
        'version': '1.0.0'
    })

@app.route('/health') def health():     return
jsonify({'health': 'healthy', 'code': 200})
 if __name__ ==
'__main__':
    app.run(host='0.0.0.0', port=5000)
```

7.3 Automated Unit Test Suite (test_app.py) devops-demo/test_app.py

```
import unittest from
app import app
 class TestApp(unittest.TestCase):
def setUp(self):         self.client =
app.test_client()
     def
test_home_route(self):
        response = self.client.get('/')
        self.assertEqual(response.status_code, 200)
self.assertIn(b'success', response.data)
     def
test_health_route(self):
        response = self.client.get('/health')
self.assertEqual(response.status_code, 200)
 if __name__ ==
'__main__':
    unittest.main()
```

8. Jenkins Job and Pipeline Configuration

Two implementation approaches were executed to demonstrate source code checkout: a Jenkins Freestyle Project and an advanced Declarative Pipeline.

8.1 Freestyle Job Configuration Procedure

1. Open Jenkins Dashboard -> Select 'New Item'.
2. Enter Item Name: 'GitHub-Jenkins-Demo' -> Select 'Freestyle project' -> Click 'OK'.
3. Under General Tab: Check 'GitHub project' and enter https://github.com/nirajpatil/devops-demo/.
4. Under Source Code Management: Select 'Git'.

- Repository URL: https://github.com/nirajpatil/devops-demo.git
- Credentials: Select configured GitHub Personal Access Token credential.
- Branch Specifier: */main

5. Under Build Steps: Add 'Execute shell' or 'Execute Windows batch command':

Build Step Commands

```
dir /s /b
python -m py_compile app.py python
-m unittest test_app.py
```

8.2 Declarative Pipeline Implementation (Jenkinsfile)

The process is codified into version-controlled Pipeline as Code using the following Jenkinsfile:

devops-demo/Jenkinsfile

```
pipeline {
agent any

    stages {
        stage('Checkout') {
steps {
                echo 'Cloning repository from GitHub...'
git branch: 'main',
                    credentialsId: 'github-pat-niraj',
                    url: 'https://github.com/nirajpatil/devops-demo.git'
            }
}
        stage('Build') {
steps {
                echo 'Compiling and verifying application syntax...'
bat 'python -m py_compile app.py'
            }
}
        stage('Test') {
steps {
                echo 'Running automated unit test suite...'
bat 'python -m unittest test_app.py'             }
        }
    }      post
{         always
{
            echo 'Pipeline execution complete.'
        }
success {
            echo 'Build succeeded and all tests passed!'
        }
failure {
            echo 'Build failed. Notifying engineering team.'
}


    }
}
```

9. Automated Trigger and Webhook Configuration

Automating the retrieval requires decoupling build execution from manual user clicks. Two trigger strategies are evaluated:

9.1 SCM Polling vs GitHub Webhooks

Trigger Strategy	Mechanism	Best Use Case
Poll SCM (H/5 * * * *)	Jenkins periodically polls GitHub API to detect commit changes.	Private intranet labs behind NAT / firewalls.
GitHub Webhook (push)	GitHub sends an instant HTTP POST payload to Jenkins on push.	Production CI/CD with publicly accessible endpoints.
Build Periodically	Executes builds on fixed cron schedules regardless of commits.	Nightly integration and regression testing.

10. Execution Workflow

The step-by-step practical sequence executed during Jenkins-GitHub integration:

Step #	Action / Procedure	Expected Verification Point

1	Create GitHub repository and push sample app.	Remote repository contains app.py, index.html, README.md.
2	Install Git & GitHub Plugins in Jenkins.	Plugins show Active under Manage Jenkins -> Plugin Manager.
3	Configure GitHub Credentials in Jenkins.	PAT credential stored with ID 'github-pat-niraj'.
4	Create 'GitHub-Jenkins-Demo' Freestyle project.	Job created and configuration dashboard opens.
5	Configure Git SCM with URL and */main branch.	Jenkins validates repository connection without red errors.
6	Add Build Step: python app.py / unittest.	Build step registered in project configuration.
7	Click 'Build Now' on Jenkins Dashboard.	Build #1 enters build queue and starts executing.
8	Inspect Console Output and Workspace files.	Git clone completes, tests pass, and status shows SUCCESS.

11. Security, Best Practices and Good Practices

Practice	CI/CD Implementation & Application
Personal Access Tokens (PAT)	Never use plaintext GitHub account passwords; issue least-privilege tokens with 'repo' scope.
Credentials Store Isolation	Store tokens inside Jenkins Credentials Provider; never commit credentials into Jenkinsfile.
Shallow Cloning	Configure shallow clone (depth: 1) for large repositories to accelerate checkout speeds and save disk.
Clean Before Checkout	Enable 'Clean before checkout' SCM extension to purge unversioned leftover build artifacts.
Timeout Boundaries	Enforce execution timeouts on build steps to prevent runaway hanging processes from locking executors.
Pipeline as Code	Commit Jenkinsfile alongside application code to version control the build pipeline alongside the application.

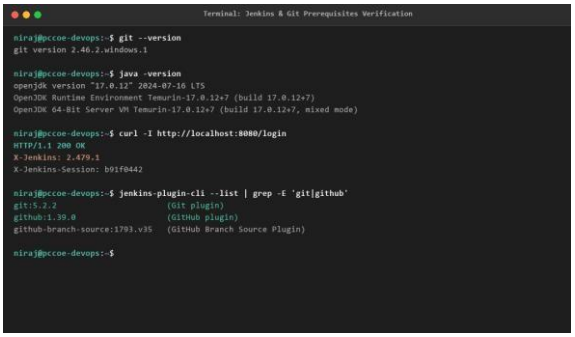
12. Practical Verification Checklist

Activity	Verification Point	Evidence to Attach
Activity	Verification Point	Evidence to Attach
Jenkins & Git Setup	Git and Java versions verified; Jenkins active on port 8080.	CLI and plugin verification screenshot (Evidence A).
Job SCM Configuration	Git repository URL and main branch configured without errors.	Jenkins job configuration screenshot (Evidence B).
Build Execution	Build #1 finishes with SUCCESS and logs Git clone details.	Console output log screenshot (Evidence C).
Workspace & Test Verification	Files present in workspace and unit test suite passes cleanly.	Workspace tree and test report screenshot (Evidence D).

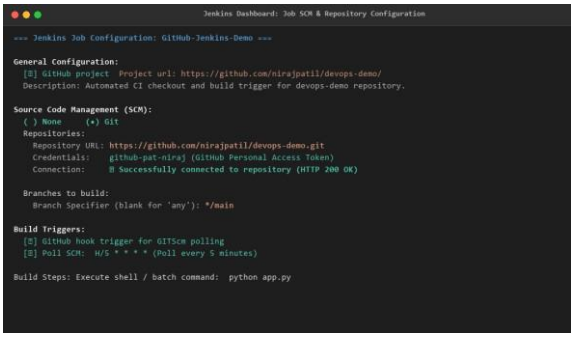
13. Practical Evidence

The captured evidence records the complete execution of Jenkins integration with GitHub, demonstrating prerequisites verification, repository SCM configuration, build execution with console logging, and workspace source verification:

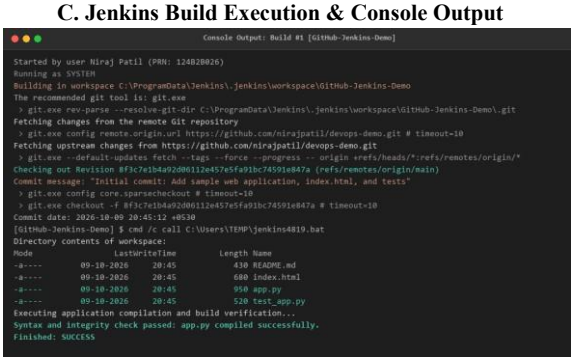
A. Environment Setup & Plugin Verification



B. GitHub Repository & SCM Configuration



C. Jenkins Build Execution & Console Output



D. Workspace Verification & Test Execution



14. Result

The laboratory assignment successfully demonstrated the end-to-end integration of Jenkins with a remote GitHub repository. Jenkins was configured with the Git plugin and authenticated credentials, a Freestyle job and Declarative Pipeline were established, and source code was automatically retrieved from the GitHub main branch upon build invocation. The checkout process, workspace structure, and automated Python test execution were thoroughly validated through the Jenkins console output with a status of SUCCESS.

15. Conclusion

Automating source code retrieval using Jenkins and GitHub forms the backbone of modern Continuous Integration (CI) practices. By delegating repository checkout, syntax compilation, and unit test execution to an automated server, engineering teams eliminate human error and ensure that mainline branches remain healthy, releasable, and audited. The practical reinforces key industry competencies in source control management, credential hygiene, declarative pipeline authoring, and automated build verification essential for agile DevOps delivery.